



REAL ESTATE MANAGEMENT SYSTEM PROPOSAL

MANAGE YOUR PROPERTY WITH EASE



System Overview

The Real Estate Management System is a digital platform designed to replace slow, paper-based tracking. It connects six types of users: Administrators, Managers, Agents, Tenants, Maintenance Staff, and Property Owners. By automating property listings, rental applications, and maintenance requests, the system saves time and helps properties stay profitable.

The Problem

Tracking rental properties manually causes several issues for the business:

- **Outdated Information:** Delays in updating property availability lead to empty units and lost money.
- **Messy Records:** Tracking maintenance and tenant history on paper makes it difficult to verify reports and expenses.
- **Slow Processes:** Manually reviewing rental applications takes too long and increases the chance of data entry errors.

What the System Does

The main goal of the system is to make everyday real estate tasks fast, smooth, and secure. Key features include:

- **Property Management:** Create property profiles, update their details, and check their status in real time.
- **Rental Applications:** Let tenants easily submit rental applications online and track their approval status.
- **Maintenance Work Orders:** Give repair staff a single place to track tickets, linking the specific property to the repair issue.
- **Reports:** Generate clean data summaries and export them as .CSV or .TXT files for management to review.

System Architecture

The application uses a standard 3-tier design to keep the system organized, scalable, and secure:

- **Presentation Layer (The Web Browser):** Built with HTML, CSS Grid, and JSPs. This is what the user sees. It automatically hides menus that a specific user role is not allowed to access.
- **Application Layer (GlassFish Server):** The "brain" of the system, built with Java. It processes the business rules and ensures all user input is safe and correct.
- **Data Layer (PostgreSQL Database):** Stores all the information securely and makes sure data relationships (like linking a tenant to a specific property) remain accurate.

User Testing Accounts

Use these accounts to test the system and verify that each user can only see and do what they are supposed to.

User Name	Role	Email	Password	What They Can Do
Boss Admin	Admin	admin@demo.com	123	Full access. Can view admin settings, edit users, and run all reports.
Diana Manager	Manager	manager@demo.com	123	High-level access. Can add properties, view reports, and track work orders. Cannot access admin settings.
Charlie Agent	Agent	agent@demo.com	123	Listing access. Can only add new properties and view the property gallery.
Bob Tenant	Tenant	tenan@demo.com	123	Self-service access. Can only submit rental applications, view their own requests, and log maintenance tickets.
Eddie Maintenance	Maintenance	staff@demo.com	123	Repair access. Can view properties and update work orders, but cannot change property details.
Alice Owner	Owner	owner@demo.com	123	Investor access. Can only view the specific properties they own.

Installation & Setup Guide

Follow these steps to set up the application server (GlassFish 4) and connect it to the database (PostgreSQL).

Prerequisites

- **JDK 1.8+ (with JAVA_HOME set):** Runs Java applications.
- **GlassFish Server 4.0:** The server where the app will live.
- **PostgreSQL 12+:** The database that stores your information.
- **postgresql-42.2.27.jar:** The driver that lets Java talk to PostgreSQL.
- **openpdf.jar:** A library needed to generate PDF reports.

Step 1: Add the Database Driver

Copy the postgresql-42.2.27.jar file and paste it into this GlassFish folder:

glassfish4/glassfish/domains/domain1/lib/ext/

(Alternatively, you can add the jar file directly to your project in your code editor).

Step 2: Set Up the Database

Open pgAdmin (or your PostgreSQL terminal). Run your database script to create the required tables (Users, Properties, MaintenanceRequests, Reports). Next, run your sample data script to insert at least 10 rows of dummy data into each table so you have something to test with.

Step 3: Start the Server

Open a terminal or command prompt and go to your GlassFish bin folder:

```
cd glassfish4/glassfish/bin/
```

Start the server by typing:

```
asadmin start-domain domain1
```

(Note: You can also start the server directly from NetBeans).

Open your web browser and go to <http://localhost:4848> to make sure the GlassFish control panel loads.

Step 4: Connect pgAdmin to the Database

1. Open pgAdmin in your browser (usually <http://127.0.0.1:5050>).
2. Right-click **Servers > Register > Server**.
3. In the setup window, enter these details:
 - **Name:** RealEstateManagementSystem
 - **Host name/address:** Your local IP (e.g., 10.252.178.5)
 - **Port:** 5432
 - **Maintenance database:** postgres
 - **Username:** RealEstateManagementSystem
 - **Password:** 123456
4. Click **Save**. If it works, you will see your new database listed on the left menu.

Step 5: Build the Application

Open NetBeans. Right-click the RealEstateManagementSystem project on the left panel and select **Clean and Build**. This compiles your code and creates a deployable .war file (e.g., RealEstateManagementSystem.war).

Step 6: Deploy the Application

1. Go back to the GlassFish Web Console (<http://localhost:4848>).
2. Click on **Applications** in the left menu.
3. Click the **Deploy** button.

4. Choose **Packaged File to be Uploaded to the Server** and browse for your generated RealEstateManagementSystem.war file.
5. Leave the context root exactly as it is (it should say /RealEstateManagementSystem).
6. Click **OK** to launch the app.